

Architectural Specification: The Compositor Wizard Pattern for Single-Page Applications

Introduction to the Compositor Paradigm

The evolution of data-dense, web-based calculation engines within single-page applications (SPAs) necessitates a departure from rigid, document-centric layouts. Modern interface architecture demands a systematic, mathematically driven approach that prioritizes component scalability, hardware-accelerated rendering performance, and uncompromising accessibility for all users¹. This specification outlines an exhaustive UI/UX architectural framework for a "Compositor Wizard"—a framework-agnostic interface pattern designed to capture highly specific domain data primitives, orchestrate them into logical blocks, and process them into synthesized, real-time outputs¹.

To illustrate the flexibility of this agnostic architecture, this document utilizes the domain of advanced brewing calculations as contextual examples. In such an application, the interface must seamlessly handle complex, interacting variables—such as alpha acid isomerization curves, malt color predictions, and water chemistry ion balancing—without exposing the user to overwhelming cognitive load⁴.

Realized through a progressively revealed accordion pattern, this architecture operates independently of any underlying backend service layer. By leveraging a lightweight, dependency-minimized technology stack consisting of Vanilla CSS, HTML5, and Alpine.js, the design achieves a strict separation of concerns. Alpine.js exclusively governs the Finite State Machine (FSM) and declarative Document Object Model (DOM) reactivity, while Vanilla CSS completely dictates the visual design system, theming algorithms, responsive layouts, and hardware-accelerated animations². This specification establishes the definitive governing principles for tokenized color semantics, fluid typography, responsive tabular data presentation, declarative state management, and strict adherence to the Web Content Accessibility Guidelines (WCAG) 2.1 and 2.2 AA standards.

The Compositor Architecture and Data Modeling

The Compositor UI architecture models complex interfaces not as a collection of static pages, but as a system of interacting, granular components that aggregate to form a comprehensive whole³. This paradigm draws inspiration from entity-component systems, wherein the user interface acts as a visual compositor that interleaves user inputs with immediate-mode logic⁷. Within the context of a calculation wizard, this paradigm is expressed through a strict tripartite hierarchy: Primitives, Blocks, and Synthesized Outputs.

The interface must visually and programmatically communicate the relationship between the smallest selectable data points and the final calculated metrics. Primitives represent the foundational, indivisible units of data input selected by the user. In a typical UI, primitives are

expressed as selectable list items, range sliders, or dropdown options. Contextually, a primitive might be an individual hop variety, such as 'Citra', possessing immutable internal properties like a specific alpha acid percentage¹. The design system must render these primitives with low visual weight until they are actively selected and engaged by the user, at which point they elevate in the visual hierarchy.

Blocks serve as the logical aggregations of these user-configured primitives. In the UI, a block is realized as a discrete surface, such as a card or a collapsible panel within an accordion structure¹. A block represents a completed, self-contained step in the wizard's overarching workflow, encapsulating its own local validation state and presentation logic. For example, a "Boil Additions" block would aggregate multiple hop primitives, organizing them by their respective boil times to facilitate downstream calculations⁵.

Synthesized Outputs represent the terminal presentation state of the compositor architecture. This layer visualizes the mathematical consequences of the configured blocks. The UI must present this synthesized data dynamically, utilizing high-contrast typography, distinct surface colors, and prominent spatial positioning to differentiate the final metrics from the raw input blocks¹. In our contextual example, the synthesized output translates raw malt weights and lovibond ratings into an estimated Standard Reference Method (SRM) color value, or translates hop additions into International Bitterness Units (IBUs)⁵.

State Management and Reactive DOM Orchestration

To maintain a strict architectural boundary between the presentation layer and the service layer, the interface relies on a frontend Finite State Machine (FSM) to govern interaction logic. Alpine.js provides the optimal mechanism for this architecture, offering declarative, proxy-based reactivity without the performance overhead or architectural complexity of a virtual DOM².

The Finite State Engine

The FSM is instantiated at the root of the wizard component using the x-data directive. This state object tracks the topological progression of the user through the accordion blocks. It is imperative that this FSM does not calculate domain-specific business logic; its sole responsibility is calculating and reflecting the presentation state of the user interface².

The core state object tracks the active step representing the currently expanded accordion block, an array of integers representing blocks that have satisfied local UI validation, and a high-water mark used to lock or unlock future steps. This mechanism enforces a sequential progression if required by the user experience flow, preventing users from accessing synthesis panels before prerequisite primitives have been configured². Furthermore, Alpine's \$watch magic method can be deployed to monitor complex nested arrays of primitives, triggering localized UI recalculations—such as updating a running subtotal within a block—without forcing a global re-render of the entire wizard¹².

Semantic Class Binding and UI Separation

A fundamental principle of this architecture is that JavaScript must never apply direct inline styling to the DOM, nor should it utilize utility-class strings that conflate state with specific visual properties. Instead, the Alpine.js FSM evaluates the reactive state object and dynamically

binds semantic CSS classes to the accordion blocks².

The user interface relies on three primary state classes to communicate status. The `.is-active` class is applied to the block currently receiving user input. This class triggers the CSS engine to expand the block's height, elevate its box shadow, and apply primary brand-color border highlights. The `.is-completed` class is applied to blocks that have successfully passed validation; this triggers CSS to render a condensed, collapsed summary view of the configured primitives, reducing vertical scrolling⁶. Finally, the `.is-locked` class is applied to future blocks that cannot yet be accessed. This class instructs the CSS to apply a reduced opacity, set pointer events to none, and desaturate internal colors to visually communicate an inactive state².

This strict separation of concerns ensures that the CSS engine retains absolute, deterministic control over the visual design system. If a block transitions to a locked state, the stylesheet automatically cascades the appropriate visual feedback without requiring JavaScript to manipulate individual color, transition, or opacity properties. This guarantees that the UI remains highly performant and drastically reduces the cognitive load required to maintain the codebase⁶.

Dynamic Array Rendering in Tabular Contexts

Configuring primitives often requires dynamic row addition, where a user sequentially adds items to a block. Alpine's `x-for` directive handles this array iteration, but doing so within a standard HTML `<table>` requires careful structural consideration. Because HTML tables enforce rigid parent-child relationships between `<tbody>` and `<tr>` elements, placing an `<template x-for="...">` directly inside a table can sometimes disrupt screen reader parsing if the DOM tree is not rendered cleanly. To mitigate this, developers can utilize CSS `display: contents` on wrapper elements generated by Alpine, ensuring that the semantic table structure remains unbroken while allowing the reactive framework to inject and remove rows dynamically as the user configures their primitives¹⁴.

The Progressively Revealed Accordion Pattern

To manage severe information density and reduce cognitive load, the Compositor Wizard utilizes a progressively revealed accordion¹. Unlike a traditional multi-page wizard that destroys spatial context upon navigation, an accordion maintains the user's location within the broader application. Users can visualize their progression through the workflow while retaining the immediate ability to review previously configured blocks¹⁶.

Disclosure Widgets Versus True Accordions

It is critical to distinguish between a simple disclosure widget and a true accordion pattern, as their interaction models and accessibility requirements differ significantly. A single disclosure widget is optimal when only one item expands, the content is secondary, and the page context remains coherent without the content being visible¹⁸. Conversely, a true accordion is a vertically stacked set of interactive headings that control multiple related sections under a unified interaction model¹⁸.

The architecture implements a hybrid expansion model governed by the Alpine.js FSM. During the active configuration phase, the accordion operates under mutually exclusive expansion.

Opening a new block programmatically forces the previously active block to collapse¹⁶. This behavior ensures that vertical screen space is conserved, preventing the user from becoming lost in a deeply scrolling page, and isolates focus entirely on the active task¹⁶. However, upon completion of the wizard, or when the user enters a dedicated review state, the UI shifts to permit concurrent expansion. This allows multiple panels to remain open simultaneously, enabling the user to compare primitive configurations across different sections of the interface¹⁶.

Animation Orchestration and Height Calculation

Animating the expansion and collapse of accordion blocks that contain dynamic data tables presents a well-documented browser rendering challenge. Standard CSS transitions cannot smoothly interpolate between a height of zero and a height of auto, leading to instantaneous, jarring layout snapping that disorients the user². To resolve this limitation, the architecture leverages Alpine's x-collapse plugin. When the state machine alters the active step, the x-collapse directive programmatically reads the target DOM node's scrollHeight property⁶. This property represents the exact pixel height of the element's visible and invisible content. Alpine applies this specific pixel value as an explicit inline style, allowing the CSS engine to execute a fluid, hardware-accelerated transition. Upon completion of the animation, Alpine removes the inline pixel constraint, restoring fluid document flow²¹. This mechanism ensures that as users add or remove primitives from an open block—thereby constantly changing the height of internal tables—the accordion container resizes seamlessly without layout recalculation penalties².

Design Systems and Token Architecture

Modern interface design relies heavily on systematic, mathematically driven design systems to ensure scalability, maintainability, and absolute consistency across multiple themes, such as light and dark modes. The Compositor Wizard utilizes a robust CSS Custom Property architecture, actively discarding hardcoded hex values in favor of a semantic token taxonomy¹.

The Token Taxonomy

The design system employs a multi-tiered token architecture, structured to entirely decouple raw visual values from their functional UI application. This taxonomy, heavily influenced by enterprise systems such as the U.S. Web Design System (USWDS), ensures a highly scalable styling environment²⁴.

Token Tier	Architectural Purpose	Naming Convention	Example
Primitive	Defines absolute raw values (colors, scales, opacities). Immutable base materials	--color-[family]-[scale]	--color-emerald-500: #1D9E75;

	representing the brand palette.		
Semantic	Defines the functional intent of a value across the application. Maps directly to Primitive tokens.	--sys-[category]-[role]	--sys-color-action-primary: var(--color-emerald-500);
Component	Defines the highly specific application of a value to a distinct element. Maps directly to Semantic tokens.	--cmp-[name]-[property]	--cmp-accordion-border: var(--sys-color-action-primary);

By strictly enforcing this taxonomy within the `:root` pseudo-class, the architecture ensures that global rebrands or theme modifications require adjusting only the primitive-to-semantic mapping¹. This cascading logic automatically propagates changes through thousands of localized component variations without requiring developers to alter specific CSS class rules²⁷.

Color Distribution and Contextual Context

To prevent visual fatigue and cognitive overload within a dense calculator interface, the UI adheres rigorously to the 60-30-10 rule for spatial color distribution¹. The dominant background, typically a pure white or dark gray, consumes sixty percent of the interface and is assigned to the default surface token. Thirty percent is dedicated to secondary surface colors, assigned to elevated tokens, which are used to establish spatial separation for active accordion blocks and summary cards. The final ten percent is reserved for high-visibility accent colors, mapped to semantic action tokens, and applied exclusively to interactive triggers, focus rings, and critical data highlights¹.

This precise allocation of color is particularly relevant when visualizing synthesized outputs. Returning to the contextual domain, a brewing calculator must often predict the SRM color of a resulting beverage based on the Morey equation, where

$$SRM = 1.4922 \times (MCU)^{0.6859}$$

²⁸. The interface can utilize the calculated

SRM output to dynamically update an inline CSS variable (e.g., `style="--calculated-srm: var(--color-srm-15)"`), instantly visualizing the mathematical result by mapping the output integer to a corresponding semantic primitive token within a specialized color scale⁹.

Dark Mode as a First-Class Citizen

The architecture supports native dark mode theming without relying on brittle JavaScript intervention. This is achieved by utilizing the `@media (prefers-color-scheme: dark)` query, or the modern CSS `light-dark()` color function, to systematically redefine the semantic token layer based on the user's operating system preferences².

Implementing dark mode requires distinct structural rules; it is never a simple color inversion¹. First, spatial depth must be communicated via lightness rather than shadow. In light themes, elevation is expressed through drop shadows. In dark themes, shadows are imperceptible. Therefore, the system redefines elevation tokens so that as a surface moves closer to the user, its background color shifts to a progressively lighter shade of dark gray¹. Second, highly saturated accent colors that perform well on white backgrounds will cause severe visual vibration and halation effects on dark surfaces. Dark mode overrides must remap semantic action colors to desaturated, pastel equivalents to maintain legibility and reduce ocular fatigue¹. Finally, pure black and pure white are strictly prohibited. Backgrounds must map to soft dark grays, while text maps to high-opacity light grays to eliminate contrast glare¹.

Typography, Scaling, and Visual Hierarchy

Typography within the Compositor Wizard is meticulously engineered to establish an immediate, undeniable visual hierarchy, explicitly separating macro-instructions found in accordion headers from the micro-data presented in primitive configuration tables¹.

The typographic system fundamentally rejects arbitrary font sizing. Instead, it relies entirely on a mathematical modular scale. By selecting a base ratio, such as 1.250, every font size is generated by multiplying the preceding size by the ratio. This mathematical relationship guarantees a harmonious, predictable visual rhythm that feels naturally balanced across the entire interface¹.

To ensure that the macro-hierarchy adapts flawlessly to varying viewport constraints without relying on hardcoded media query breakpoints, the CSS relies on fluid typography utilizing the `clamp()` function. A declaration such as `font-size: clamp(1.125rem, 2vw + 1rem, 1.5rem);` permits headings to scale smoothly and continuously along a defined continuum between mobile and desktop environments¹.

Visual hierarchy is established not merely by enlarging text, but by manipulating font weight and color contrast. Secondary instructions and metadata maintain the same font size as primary data but utilize a lighter font weight paired with a muted secondary text color token. This technique preserves highly valuable horizontal screen real estate while clearly communicating subordinate importance¹.

Tabular Numerals and Data Jitter

For all data grids containing calculated primitives, financial figures, or statistical outputs, the architecture strictly mandates the application of the `font-variant-numeric: tabular-nums;` CSS property².

This property forces proportionally spaced typefaces to utilize monospaced, fixed-width character glyphs exclusively for numeric digits. In an application where a state machine is rapidly recalculating and updating DOM nodes based on user slider inputs or text entry, standard proportional numbers will cause severe horizontal jitter, making the data nearly

impossible to track visually. Tabular numerals ensure that decimal points align perfectly down columns, drastically reducing cognitive load when users are scanning complex outputs, such as the exact milligram-per-liter concentrations of Calcium, Magnesium, and Sulfates in a detailed water chemistry profile table².

Layout Orchestration and Container Queries

The internal blocks of the Compositor Wizard house highly dense data tables and complex configuration grids. Relying on traditional viewport-based media queries (`@media`) is entirely insufficient for this architecture, as an accordion block may be embedded within vastly different layout contexts—such as occupying the full width of a mobile screen, or being constrained within a narrow secondary sidebar on a desktop monitor³².

To achieve true component modularity, the architecture utilizes CSS Container Queries (`@container`). The accordion panel wrapper is explicitly designated as a containment context using the `container-type: inline-size;` property³².

All internal user interface components—such as primitive selection grids and dense data tables—query the dimensions of their specific parent accordion panel, rather than the global browser window. This isolation of component logic ensures the wizard remains strictly application-agnostic, capable of being seamlessly embedded into any external layout architecture without breaking its internal responsive behavior³². When the container width drops below a specified threshold, internal flexbox or grid layouts can gracefully reflow from horizontal rows into vertical stacks.

Responsive Tabular Presentation

Standard HTML tables are inherently resistant to responsive design due to their rigid, multi-column structural enforcement³⁶. Because a calculation wizard relies heavily on tabular data to display the interconnected properties of configured primitives, the interface must implement robust responsive patterns to prevent data obfuscation.

Data Grids Versus Data Tables

A common and highly destructive architectural error in complex web applications is aggressively applying the WAI-ARIA `role="grid"` attribute to standard tabular data. The ARIA grid pattern is a specialized, composite interactive widget designed strictly for spreadsheet-like applications where users must navigate two-dimensionally across editable cells using arrow keys³⁷.

Implementing `role="grid"` without simultaneously building the highly complex JavaScript event listeners required to manage a roving tabindex effectively traps keyboard users, destroying the usability of the interface³⁸. Therefore, the Compositor Wizard deliberately utilizes standard, semantic HTML `<table>` elements for its primitive lists and synthesized output ledgers. Standard semantic tables naturally include all focusable child elements, such as input fields and delete buttons, in the standard document tab sequence, ensuring predictable and robust sequential navigation for assistive technologies without demanding unnecessary engineering overhead³⁷.

Feature	HTML <code><table></code>	ARIA <code>role="grid"</code>
---------	---------------------------------	-------------------------------

Primary Use Case	Reading data, simple sequential inputs.	Spreadsheet editing, complex 2D navigation.
Keyboard Navigation	Native Tab sequence through interactive elements.	Custom JavaScript arrow-key roving tabindex.
Development Cost	Low. Semantic HTML provides native accessibility.	Extremely High. Requires manual focus management.
Assistive Tech parsing	Native table commands (column/row headers read).	Requires manual implementation of aria-colindex.

The Horizontal Scroll Paradigm

To manage standard HTML tables on constrained viewports, the architecture mandates the Horizontal Scroll method as the primary paradigm³⁶. This approach preserves the mathematical alignment of the data columns while accommodating narrow screens.

The <table> element is wrapped in a discrete container applying overflow-x: auto;. To prevent the user from losing their orientation as they scroll horizontally through primitive attributes, the primary identifying column utilizes position: sticky; left: 0;³⁶. A subtle, token-driven drop shadow is applied to the right edge of this sticky column, visually indicating to the user that obscured content is passing beneath it³⁶.

Decoupling Complex Interactions

When utilizing horizontally scrolling tables, relying on column headers for sorting interactions becomes a severe user experience anti-pattern on mobile devices, as the desired sorting metric may be scrolled entirely out of the visible viewport³⁶.

The architecture solves this by decoupling the sorting mechanism from the table headers entirely. Alpine.js is utilized to render a global dropdown control positioned explicitly above the table within the accordion block. When the user selects a metric, the state machine reacts, sorts the internal data array, and natively updates the DOM table representation. This provides a generous, touch-friendly hit target and ensures the sorting context remains permanently visible regardless of the table's scroll position².

Similarly, calculators inherently produce summary aggregate data. Forcing a mobile user to scroll horizontally to the final column of a table simply to view a synthesized "Grand Total" introduces severe friction³⁶. The architecture mitigates this by extracting critical aggregate outputs from the table and rendering them as highly legible metric cards positioned directly above the table controls. The table below then serves purely as a detailed ledger to

mathematically justify the numbers displayed in the decoupled cards³⁶.

Accessibility Standards and Semantic Integrity

A compositor wizard represents a highly complex, intensely interactive interface. Compliance with the Web Content Accessibility Guidelines (WCAG) 2.1 and 2.2 AA is a foundational architectural requirement, not a post-development remediation effort. The UI layer must expose its state flawlessly and deterministically to assistive technologies.

WAI-ARIA Accordion Semantics

An accordion that relies solely on visual CSS state changes is completely invisible to users relying on screen readers. The HTML structure must adhere strictly to the WAI-ARIA Authoring Practices Guide (APG) for disclosure widgets¹⁸.

The accordion header trigger cannot be implemented as a styled `<div>` with a click handler. It must be a native `<button>` element, wrapped inside a semantic heading tag. The heading tag establishes the document outline, allowing screen reader users to quickly jump between thematic blocks, while the native `<button>` provides built-in keyboard focus and interaction events out of the box¹⁶.

Crucially, the trigger button must implement the `aria-expanded` attribute. When the Alpine state machine toggles the block's active status, it must simultaneously toggle `aria-expanded="true"` or `aria-expanded="false"`. This provides the critical programmatic state change announcement to assistive technology¹⁸. Furthermore, the button must declare an `aria-controls` attribute whose value identically matches the ID of the collapsible accordion panel, explicitly binding the interactive trigger to the content region it reveals¹⁸. When a block is collapsed, its internal content must be completely removed from the accessibility tree via the `hidden` attribute to prevent keyboard users from tabbing into invisible inputs¹⁸.

Target Sizing and Spatial Geometry (WCAG 2.5.8)

Complex configuration grids often suffer from densely packed interactive elements. The architecture enforces strict compliance with the newly introduced WCAG 2.2 Target Size requirements.

The minimum standard dictates that every interactive target—including buttons, dropdown toggles, and table row expanders—must measure at least 24 by 24 CSS pixels⁴³. However, in highly constrained data tables where enforcing a 24-pixel physical boundary breaks the layout, the architecture invokes the spacing exception. An interactive target may physically render smaller than 24 pixels provided that an imaginary 24-pixel diameter circle centered on the target does not intersect with the 24-pixel circle of any adjacent target⁴³.

Target Size	Gap Required	Total Area Calculation	Compliance Status
24x24 px	0 px	24px diameter	Pass (Meets minimum size) ⁴⁵

20x20 px	4 px	24px diameter	Pass (Via spacing exception) ⁴⁵
16x16 px	8 px	24px diameter	Pass (Via spacing exception) ⁴⁵
20x20 px	0 px	20px diameter	Fail (Intersection occurs) ⁴⁵

For all primary macro-interactions, specifically the accordion header triggers and global wizard progression buttons, the interface mandates a minimum target size of 44 by 44 CSS pixels. This enhanced size gracefully accommodates physical finger dimensions on touch interfaces, aligning with WCAG 2.5.5 AAA recommendations and major platform human interface guidelines¹⁶.

Contrast and Focus Management

The architecture utilizes programmatic color token generation to guarantee WCAG contrast compliance. All text expressing primitive data or synthesized outputs must meet a strict 4.5:1 contrast ratio against its surface background¹. Furthermore, non-text UI component boundaries, such as the borders of text inputs or directional sorting icons, must meet a 3:1 contrast ratio against their adjacent colors to ensure users with low vision can accurately identify control boundaries⁴⁷.

Focus indicators are mandatory for keyboard navigation, but persistent focus rings can create visual clutter for pointer device users. The architecture utilizes the CSS :focus-visible pseudo-class. This modern selector ensures that custom, high-contrast focus outlines are applied exclusively when the browser detects keyboard-based navigation heuristics, preserving a pristine aesthetic for mouse users while fully satisfying accessibility mandates⁵⁰.

Dynamic Feedback and Assistive Technologies

The defining characteristic of a compositor calculator is its ability to immediately synthesize complex outputs based on the manipulation of internal primitives. For example, adjusting the quantity of a roasted malt input inside a grain bill block will instantly recalculate the total batch color output⁹. Similarly, toggling the IBU calculation algorithm from Tinseth to Rager will instantly alter the bitterness output based on differing utilization curves and gravity adjustment factors⁵.

While these reactive DOM updates are immediately visually apparent, they must be programmatically announced to screen reader users without aggressively stealing their focus or interrupting their current interaction flow.

Orchestrating ARIA Live Regions

The architecture employs WAI-ARIA Live Regions to gracefully manage dynamic recalculation announcements⁵⁴. The container holding the synthesized mathematical outputs is designated with the aria-live="polite" attribute⁵⁷.

The polite configuration explicitly instructs the assistive technology to wait until the user has

paused their current task—such as finishing typing a number or completing a slider drag—before audibly announcing the updated totals⁵⁷. This provides critical feedback without causing disorientation. Furthermore, the `aria-atomic="true"` attribute ensures that when a single integer changes within a string, the screen reader announces the entire region's context, rather than disjointedly reading an isolated number⁵⁶.

The architecture strictly reserves the `aria-live="assertive"` attribute for critical, workflow-blocking validation errors that require immediate attention⁵⁹. Utilizing the assertive configuration for routine mathematical updates results in severe user experience degradation, as it violently interrupts the user's ongoing interactions.

The Necessity of State Debouncing

Because the Alpine.js reactive engine updates the DOM instantaneously, rapidly dragging a range slider to adjust a primitive's quantity could trigger dozens of consecutive `aria-live` announcements, severely overwhelming the assistive technology queue and rendering the interface unusable for non-sighted users.

To mitigate this architectural hazard, the UI requires the implementation of debounced state bindings. By utilizing modifiers such as Alpine's `.debounce` on the `x-model` directive, the architecture ensures that the DOM node residing within the `aria-live` region only updates after the user has paused their interaction for a predetermined threshold (typically 300 to 500 milliseconds). This results in a single, clean announcement of the final, synthesized mathematical state, preserving both performance and accessibility.

Conclusion

The Compositor Wizard architecture establishes a rigorous, mathematically driven framework for developing complex, data-dense calculation interfaces. By strictly isolating the presentation state within Alpine.js and delegating all visual rendering to a semantic, CSS Custom Property-driven design system, the architecture ensures unparalleled scalability, maintainability, and hardware-accelerated performance.

The adherence to progressive disclosure via a mutually exclusive accordion pattern respects the user's cognitive capacity by effectively managing information density. Concurrently, the deployment of CSS Container Queries and horizontal overflow mechanics guarantees that dense, tabular primitive data remains perfectly legible across any viewport constraint. Above all, the deep integration of WCAG 2.2 AA target sizing, precise WAI-ARIA disclosure semantics, and debounced `aria-live` regions ensures that the interface remains universally accessible and predictable. This application-agnostic blueprint provides a highly resilient foundation capable of supporting any domain-specific data configuration, gracefully transforming raw input primitives into synthesized, actionable outputs.

Works cited

1. `color_theming_and_typography.md`
2. `color_typography_state_management.md`
3. Introducing design systems - Webflow,
<https://uploads-ssl.webflow.com/609ce44d3dfdab98b5019cf9/614360ea598313b>

[1789c3d1d_DesignSystemsHandbook.pdf](#)

4. Brewing Water Chemistry: A Practical Guide For Craft Brewers, <https://beerrepublic.eu/blogs/news/brewing-water-chemistry>
5. How to Calculate IBUs for Any Hop Schedule - BrewCalc, <https://brewcalc.io/blog/how-to-calculate-ibu>
6. css_only_strategy.md
7. Building a UI framework - Hixie, <https://software.hixie.ch/ui-frameworks.pdf>
8. IBU Calculator - Homebrew Academy, <https://homebrewacademy.com/ibu-calculator/>
9. What Is SRM and How Does It Measure Beer Color?, <https://dointhemoost.org/guides/what-is-srm-and-how-does-it-measure-beer-color/>
10. Mastering Alpine.js: Advanced Patterns and Techniques - Medium, <https://medium.com/@satnammca/mastering-alpine-js-advanced-patterns-and-techniques-023e5de40fa8>
11. Blog - ioDropLet, <https://iodroplet.com/blog/>
12. Alpine.js - sync computed field back to model - Stack Overflow, <https://stackoverflow.com/questions/73404398/alpine-js-sync-computed-field-back-to-model>
13. Design Systems Blueprint v1.0 — 24-Layer CSS Architecture, <https://deterministiccore.com/design-systems/>
14. Alpine.js: nested x-for in a table: how to wrap multiple elements, <https://stackoverflow.com/questions/73784230/alpine-js-nested-x-for-in-a-table-how-to-wrap-multiple-elements-without-breaki>
15. Migrating from Alpine.js | WildflowerJS, <https://www.wildflowerjs.com/docs/alpine-migration/>
16. general_accordion_principles.md
17. Accordion Pattern | UX Patterns for Developers, <https://uxpatterns.dev/patterns/content-management/accordion>
18. Accordion Accessibility: A Practical WCAG Compliance Guide, <https://www.adacompliancepros.com/blog/accordion-accessibility>
19. Patterns | APG | WAI - W3C, <https://www.w3.org/WAI/ARIA/apg/patterns/>
20. Accordion Pattern (Sections With Show/Hide Functionality) | APG | WAI, <https://www.w3.org/WAI/ARIA/apg/patterns/accordion/>
21. Slide up/down animation with AlpineJS - Hussein Al Hammad, <https://hussein-alhammad.com/blog/2023/03/alpine-slide-up-down-animation/>
22. Animate to height: auto; (and other intrinsic sizing keywords) in CSS, <https://developer.chrome.com/docs/css-ui/animate-to-height-auto>
23. Collapse — Alpine.js, <https://alpinejs.dev/plugins/collapse>
24. Migrating to USWDS 2.0, <https://designsystem.digital.gov/documentation/migration-v2/>
25. Design tokens - VA.gov Design System - Veterans Affairs, <https://design.va.gov/foundation/design-tokens/>
26. Design Tokens and Web Components | U-M Library Design System, <https://design-system.lib.umich.edu/design-and-development/design-tokens-and>

[-web-components/](#)

27. The developer's guide to design tokens and CSS variables - Penpot,
<https://penpot.app/blog/the-developers-guide-to-design-tokens-and-css-variables/>
28. Beer-colour - Blue Bottle Brewing,
<https://bluebottlebrewing.co.uk/design/beer-colour/>
29. Calculating SRM - Highwoods Brewing,
<http://www.highwoodsbrewing.com/srm-color.php>
30. color-scheme and light-dark() - Medium,
<https://medium.com/@projectluis/color-scheme-and-light-dark-e98c041742f9>
31. Design Tokens To Dark Mode - Frank Congson,
<https://frankcongson.com/blog/design-tokens-to-dark-mode/>
32. CSS Container Queries: Building Truly Responsive Web Components,
<https://www.c-sharpcorner.com/article/css-container-queries-building-truly-responsive-web-components>
33. CSS Container Queries: Write Truly Responsive Components (Finally!),
<https://dev.to/hamidrazadev/css-container-queries-write-truly-responsive-components-finally-3bl4>
34. container-type - CSS-Tricks,
<https://css-tricks.com/almanac/properties/c/container-type/>
35. CSS container queries - MDN Web Docs,
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries
36. responsive_html_tables.md
37. Grid (Interactive Tabular Data and Layout Containers) Pattern - W3C,
<https://www.w3.org/WAI/ARIA/apg/patterns/grid/>
38. Accessible Data Grid Guide: role=grid & Keyboard Nav,
<https://accessibility.build/guides/accessible-data-grid>
39. ARIA Grid As an Anti-Pattern - Adrian Roselli,
<http://adrianroselli.com/2020/07/aria-grid-as-an-anti-pattern.html>
40. ARIA Grid: Supporting nonvisual layout and keyboard traversal,
<https://engineering.fb.com/2017/03/28/web/aria-grid-supporting-nonvisual-layout-and-keyboard-traversal/>
41. Making Accordions and Expandable Sections Accessible - Quietramp,
<https://quietramp.com/notes/accessible-accordions-expandable-sections-ecommerce.html>
42. ARIA Patterns & Components | Accessibility - The University of Arizona,
<https://accessibility.arizona.edu/web-apps/aria-patterns>
43. What Is the WCAG 2.5.8 Target Size Minimum and How Do You,
<https://testparty.ai/blog/wcag-target-size-guide>
44. Understanding Success Criterion 2.5.8: Target Size (Minimum) | WAI,
<https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
45. Target Size (Minimum) - WCAG 2.2 AA, explained,
<https://wcag22aa.org/new-criteria/target-size/>
46. Foundations: target sizes - TetraLogical,

- <https://tetralogical.com/blog/2022/12/20/foundations-target-size/>
47. Color Contrast Requirements: WCAG Guide for Designers and, <https://testparty.ai/blog/color-contrast-requirements>
 48. Accessibility | Color & Type - UCLA Brand Guidelines, <https://brand.ucla.edu/fundamentals/accessibility/color-type>
 49. Exploring WCAG 2.1 — 1.4.11 Non-text Contrast - Knowbility, <https://knowbility.org/blog/2018/wcag21-1411contrast>
 50. focus-visible CSS pseudo-class - MDN Web Docs, <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/:focus-visible>
 51. Standardizing Focus Styles With CSS Custom Properties, <https://css-tricks.com/standardizing-focus-styles-with-css-custom-properties/>
 52. Understanding the Difference Between `:focus` and ` ` - This Dot Labs, <https://www.thisdot.co/blog/understanding-the-difference-between-focus-and-focus-visible-in-css>
 53. A Modern Method for Calculating IBUs - Brew Your Own, <https://byo.com/articles/ibu/>
 54. aria-live - Accessibility.com, <https://www.accessibility.com/glossary/aria-live>
 55. Accessible notifications with ARIA Live Regions (Part 1), <https://www.sarasoueidan.com/blog/accessible-notifications-with-aria-live-regions-part-1/>
 56. Introduction-to-ARIA-training.pdf - CDN, <https://bpb-us-w2.wpmucdn.com/voices.uchicago.edu/dist/9/2108/files/2019/04/introduction-to-ARIA-training.pdf>
 57. Fixing aria-live in Angular, React, and Vue | k9n.dev, <https://k9n.dev/de/blog/2025-11-aria-live/>
 58. ARIA Cheat Sheet for React: aria-label, Roles & Attributes - thefrontkit, <https://thefrontkit.com/blogs/aria-attributes-cheat-sheet-for-react>
 59. Live regions must be able to announce status messages - EqualWeb, <https://www.equalweb.com/wiki/status-messages>
 60. ARIA | ASU Digital Accessibility, <https://accessibility.asu.edu/articles/aria>